SMART CONTRACT DEVELOPMENT ASSIGNMENT

Design, Deploy & Test 15 Solidity Smart Contracts
using Remix IDE and Ganache

Course	Blockchain / Solidity Development
Toolchain	Solidity ^0.8.x	Remix IDE	Ganache	Git & GitHub
Deliverable	Personal GitHub repository (link shared with instructor)
Total Marks	100
Contracts	15 (Basic → Expert)

1. Assignment Overview

In this assignment you will design, write, deploy and test **15 smart contracts in Solidity**, ordered from very basic to expert level. Every contract must be compiled in **Remix IDE**, deployed and tested against **Ganache** (your personal local blockchain), and finally pushed to **your own GitHub repository**. At the end you will submit only **one thing: the link to your GitHub repository**.

Learning outcomes — after completing this assignment you will be able to:

- Write Solidity contracts of increasing complexity (state, functions, modifiers, events, mappings, structs, inheritance).
- Compile and deploy contracts using Remix IDE.
- Connect Remix to a Ganache local blockchain and test contracts with real transactions.
- Read transaction details, logs/events and account balances in Ganache.
- Use Git & GitHub to version, document and publish your code professionally.

Important rules:

- Use **Solidity ^0.8.x** (pragma solidity ^0.8.0 or higher) for ALL contracts — 0.8+ has built-in overflow checks.
- Every contract must be **tested on Ganache**, not only on the Remix VM.
- Every contract needs **at least 2 screenshots** of successful testing (see Section 6).
- Do NOT copy-paste full solutions from the internet. Understand every line — you may be asked to explain your code in a viva.
- Plagiarism between students = zero marks for both.

2. Tool Setup (do this first)

2.1 Remix IDE (no installation needed)

- Open **https://remix.ethereum.org** in your browser (Chrome/Edge/Firefox recommended).
- In the File Explorer, create a workspace named **Assignment15**.
- Inside it, create one folder per contract: **01_Storage.sol**, **02_Greeting.sol**, … (names are given in Section 5).
- The Solidity Compiler tab: choose compiler version 0.8.x and click **Compile**. Fix all errors and warnings where possible.

2.2 Ganache (your personal local blockchain)

- Download the **Ganache GUI** from https://archive.trufflesuite.com/ganache/ (or install CLI: **npm install -g ganache**).
- Create a new workspace: **QUICKSTART Ethereum**.
- Note the RPC server address at the top: it must be **HTTP://127.0.0.1:7545**.
- Ganache gives you **10 accounts**, each preloaded with **100 ETH**. Note the private keys (key icon) — you will import them into MetaMask for testing.

2.3 Connect Remix to Ganache (MANDATORY for testing)

There are two accepted methods — use **Method A** for all your testing so transactions appear in Ganache:

```
Method A (Web3 Provider – recommended):
  1. Start Ganache (RPC = HTTP://127.0.0.1:7545).
  2. In Remix: Deploy & Run tab → Environment → select 'Custom - External Http Provider'.
  3. URL = http://127.0.0.1:7545 → OK.
```

```
  4. The Accounts dropdown now shows Ganache accounts. Deploy here.
  5. Every deploy/transaction now appears inside the Ganache 'BLOCKS' and 'TRANSACTIONS' t
abs.

Method B (MetaMask + Ganache — alternative):
  1. Install MetaMask in the browser → import an account using a Ganache private key.
  2. Add network manually: RPC http://127.0.0.1:7545, Chain ID 1337 (or 5777), Currency ET
H.
  3. In Remix choose Environment = 'Injected Provider - MetaMask' and deploy.
```

Sanity check before you start Contract 01: deploy anything in Remix, then open Ganache — a new block with a contract-creation transaction must appear. If it does not, your connection is wrong.

3. GitHub Setup (for students who do NOT have a GitHub account)

Follow these steps exactly, in order:

Step 1 — Create your GitHub account

- Go to **https://github.com** → click **Sign up**.
- Use your college/personal email, choose a **professional username** (e.g. vikram-puri, not coolboy123).
- Verify your email address.

Step 2 — Install Git on your computer

```
Windows :  https://git-scm.com/download/win   → install with default options (installs Gi
t Bash)
macOS   :  brew install git      (or https://git-scm.com/download/mac)
Linux   :  sudo apt install git

Verify :  open a terminal / Git Bash and run:
           git --version
```

Step 3 — Configure Git (one time only)

```
git config --global user.name  "Your Name"
git config --global user.email "your-github-email@example.com"
```

Step 4 — Create the repository ON GitHub

- On GitHub click the **+** icon (top-right) → **New repository**.
- Repository name: **solidity-smart-contracts-assignment** (exactly this).
- Set it to **Public**. Tick “Add a README file”. Add a .gitignore for nothing specific — leave default.
- Click **Create repository**.

Step 5 — Connect your local folder to GitHub and push

```
# 1. Make a local folder and put ALL your .sol files + screenshots inside it
mkdir solidity-smart-contracts-assignment
cd solidity-smart-contracts-assignment

# 2. Copy your contracts and screenshots here, then:
git init
git add .
git commit -m "Add 15 tested smart contracts with screenshots"

# 3. Link to YOUR repository (replace YOUR-USERNAME):
```

```
git remote add origin https://github.com/YOUR-USERNAME/solidity-smart-contracts-assignment
.git
git branch -M main
git push -u origin main
```

When GitHub asks for a password, paste a **Personal Access Token**, not your account password: GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token → tick the **repo** scope.

Step 6 — Verify: refresh your repository page on github.com. All 15 .sol files, the README and the screenshots folder must be visible. This page is what you will submit.

4. Required Repository Structure

Your repository MUST follow this exact structure — marks are deducted for missing files:

```
solidity-smart-contracts-assignment/
|
|-- README.md                      <- filled properly (template below)
|-- contracts/
|   |-- 01_Storage.sol
|   |-- 02_Greeting.sol
|   |-- 03_Counter.sol
|   |-- 04_Ownable.sol
|   |-- 05_SimpleBank.sol
|   |-- 06_Voting.sol
|   |-- 07_MyToken.sol
|   |-- 08_Crowdfunding.sol
|   |-- 09_Lottery.sol
|   |-- 10_TimeLockVault.sol
|   |-- 11_Escrow.sol
|   |-- 12_SimpleNFT.sol
|   |-- 13_EnglishAuction.sol
|   |-- 14_MultisigWallet.sol
|   |-- 15_StakingRewards.sol
|-- screenshots/
|   |-- 01_compile.png ... 15_test2.png   (min. 2 per contract, 30 total)
|-- report/
    |-- REPORT.md                  <- short report per contract (see Section 6)
```

README.md template — copy this and fill it in:

```
# Solidity Smart Contract Assignment
**Name   :**
**Roll No:**
**Email  :**

| #  | Contract            | Level   | Tested on Ganache | Screenshots |
|----|---------------------|---------|-------------------|-------------|
| 01 | Storage             | Basic   | YES               | 01_x.png    |
| ...| ...                 | ...     | ...               | ...         |

## How to run
1. Open Remix -> paste contract -> compile with 0.8.x
2. Ganache QUICKSTART on 127.0.0.1:7545
3. Remix: Environment = Custom External Http Provider
4. Deploy and test per the checklist in each contract file
```

5. The 15 Contracts — Design Specifications

For every contract below you must: (a) write the full Solidity code yourself, (b) implement **all** listed requirements, (c) compile in Remix, (d) deploy on Ganache via Web3 Provider, (e) test every function including error cases, (f) take screenshots. A **skeleton with function signatures** is given — you design the internals. Difficulty rises from 01 to 15.

CONTRACT 01 — Simple Storage (file: 01_Storage.sol)

LEVEL: Basic | MARKS: 3 | EST. TIME: 30 min

Problem: Your "hello world" of blockchain. Store and update a single unsigned integer on-chain.

Required design:

```
contract Storage {
    uint256 private storedData;          // state variable
    constructor(uint256 _init) { ... }   // set initial value at deploy
    function set(uint256 _value) public { ... }
    function get() public view returns (uint256) { ... }
}
```

- **set()** must update storedData and emit an event **ValueSet(address indexed who, uint256 newValue)**.
- **get()** is a view function (no gas when called).

  Test on Ganache: deploy with initial value 42 → get() shows 42 → set(100) from Account 1 → confirm a transaction appears in Ganache → get() shows 100 → open the transaction in Ganache and find your event log.

CONTRACT 02 — Greeting Contract (file: 02_Greeting.sol)

LEVEL: Basic | MARKS: 4 | EST. TIME: 40 min

Problem: A contract that stores a greeting message; only the owner may change it, anyone may read it.

```
contract Greeting {
    string private greeting;
    address public owner;
    constructor(string memory _g) { ... }        // deployer = owner
    function setGreeting(string memory _g) public { ... }  // owner-only, revert if not
    function greet() public view returns (string memory) { ... }
    function resetGreeting() public { ... }      // owner-only, back to default 'Hello, Wo
rld!'
}
```

- Use **require()** with a clear error message for the owner check.
- Emit **GreetingChanged(address changer, string newGreeting)** on every change.

  Test: greet() returns default → setGreeting('GM Ganache') from owner works → switch to Account 2 in Remix and call setGreeting → the transaction must **revert** with your error message → screenshot both cases.

CONTRACT 03 — Counter with Events (file: 03_Counter.sol)

LEVEL: Basic | MARKS: 5 | EST. TIME: 40 min

Problem: A counter that can only move within bounds [0, MAX]. You must protect the boundaries.

```
contract Counter {
    uint256 public count;
    uint256 public constant MAX = 100;
    function increment() public { ... }   // revert 'Max limit reached' at 100
```

```
    function decrement() public { ... }   // revert 'Count is zero' at 0
    function reset() public { ... }       // sets 0, resets only via explicit call
    // events: Incremented(address, uint256 newCount), Decremented(...), Reset(address)
}
```

Test: loop increment to 100 (or set count via constructor start), prove the revert at the limit, prove decrement revert at zero, verify all events in Ganache logs.

CONTRACT 04 — Ownable + Inheritance (file: 04_Ownable.sol)

LEVEL: Medium | MARKS: 6 | EST. TIME: 1 hr

Problem: Build a reusable ownership base contract and a child contract that inherits it. This is the foundation pattern of real DeFi contracts.

```
contract Ownable {
    address public owner;
    event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);
    constructor() { ... }                        // msg.sender = owner
    modifier onlyOwner() { ... }                 // revert 'Not the owner' if msg.sender !=
 owner
    function transferOwnership(address newOwner) public onlyOwner { ... }
                                              // revert if newOwner == address(0)
}

contract ProtectedVault is Ownable {          // inherits Ownable
    uint256 private treasure;
    function deposit() public payable { ... }          // anyone can deposit ETH
    function withdraw(address to, uint256 amt) public onlyOwner { ... }
    function contractBalance() public view returns (uint256) { ... }
}
```

- Explain (in your report) why modifiers save code duplication.
- Use **payable** correctly and check that withdraw() cannot send more than the balance.

  Test: deposit 1 ETH from Account 1 → Account 2 calls withdraw → revert → owner transfers ownership to Account 2 → now Account 2 can withdraw → verify ETH balance change of both accounts in Ganache.

CONTRACT 05 — Simple Bank (file: 05_SimpleBank.sol)

LEVEL: Medium | MARKS: 6 | EST. TIME: 1 hr

Problem: Each user has their own on-chain balance; they can deposit and withdraw ETH safely.

```
contract SimpleBank {
    mapping(address => uint256) private balances;
    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);
    function deposit() public payable { ... }            // require msg.value > 0
    function withdraw(uint256 amount) public { ... }     // check balance, then send ETH
    function myBalance() public view returns (uint256) { ... }
    function bankTotal() public view returns (uint256) { ... }  // address(this).balance
}
```

- You MUST implement the **Checks-Effects-Interactions** pattern: update the mapping **before** sending ETH, to prevent re-entrancy. Explain this in the report.
- Use **call{value: amount}('')** for the transfer and handle failure.

  Test: deposit 2 ETH from Account 1 and 3 ETH from Account 2 → balances correct → try withdrawing 10 ETH → revert → withdraw 1 ETH → verify Ganache account balances changed exactly 1 ETH minus gas.

CONTRACT 06 — Voting System (file: 06_Voting.sol)

LEVEL: Medium | MARKS: 7 | EST. TIME: 1.5 hrs

Problem: A chairperson deploys an election with named candidates. Registered voters vote once. The contract can declare the winner.

```
struct Candidate { string name; uint256 voteCount; }
struct Voter    { bool authorized; bool voted; uint256 voteIndex; }

Candidate[] public candidates;
mapping(address => Voter) public voters;
address public chairperson;

constructor(string[] memory candidateNames) { ... }   // chairperson = deployer
function authorize(address voter) external onlyChairperson { ... }
function vote(uint256 candidateIndex) external { ... }
    // revert if not authorized, already voted, or invalid index
function totalVotes(uint256 candidateIndex) external view returns (uint256) { ... }
function winnerName() external view returns (string memory) { ... }  // loop to find max
```

Test: add 3 candidates ('Alice','Bob','Carol') → authorize Accounts 1–3 → each votes → Account 1 tries to vote twice → revert → unauthorized Account 4 tries to vote → revert → winnerName() is correct.

CONTRACT 07 — MyToken (ERC-20 style) (file: 07_MyToken.sol)

LEVEL: Medium | MARKS: 7 | EST. TIME: 1.5 hrs

Problem: Create your own fungible token without importing OpenZeppelin — write the core ERC-20 behaviour yourself.

```
contract MyToken {
    string public name;      // e.g. 'VikToken'
    string public symbol;    // e.g. 'VKT'
    uint8  public constant decimals = 18;
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;
    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);
    constructor(string memory _n, string memory _s, uint256 _supply) { ... }  // mint all
to deployer
    function transfer(address to, uint256 value) public returns (bool) { ... }
    function approve(address spender, uint256 value) public returns (bool) { ... }
    function transferFrom(address from, address to, uint256 value) public returns (bool) {
 ... }
}
```

- transferFrom must spend from the **allowance** set by approve — this two-step flow is how DEXes work.
- Revert with clear messages on insufficient balance / insufficient allowance.

Test: deploy with 1,000,000 supply → Account 1 transfers 500 to Account 2 → balances update → Account 2 approves Account 3 for 200 → Account 3 transferFrom succeeds → second transferFrom of 200 reverts (allowance exhausted).

CONTRACT 08 — Crowdfunding / Fundraiser (file: 08_Crowdfunding.sol)

LEVEL: Advanced | MARKS: 8 | EST. TIME: 2 hrs

Problem: A campaign with a goal and a deadline. If the goal is met before the deadline the owner claims the funds; otherwise contributors get refunds.

```
struct Contribution { address contributor; uint256 amount; }
address payable public owner;
```

```
uint256 public goal;            // in wei
uint256 public deadline;        // block.timestamp based
uint256 public raised;
bool public claimed;
mapping(address => uint256) public contributions;

constructor(uint256 _goal, uint256 _durationSeconds) { ... }
function contribute() external payable { ... }
    // revert if past deadline, if amount == 0; update raised & contributions
function getRemainingTime() public view returns (uint256) { ... }
function claimFunds() external onlyOwner { ... }   // only after deadline & goal reached
function refund() external { ... }                 // only after deadline & goal FAILED
    // sends the caller their contribution back, zero their mapping, one-shot safe
```

Test (run both paths!): Campaign A: reach the goal → owner claims → contract balance 0. Campaign B: miss the goal → contributors refund → try refund twice → second must fail. Verify all ETH movements in Ganache.

CONTRACT 09 — Lottery (file: 09_Lottery.sol)

LEVEL: Advanced | MARKS: 7 | EST. TIME: 1.5 hrs

Problem: Players enter by paying exactly the entry fee; the manager picks a winner who receives the full pot.

```
address payable[] private players;
address public manager;           // deployer
uint256 public entryFee;
uint256 public minPlayers;
function enter() public payable { ... }   // exact fee, manager cannot enter
function playersCount() public view returns (uint256) { ... }
function pickWinner() public onlyManager { ... }
    // require >= minPlayers
    // pick pseudo-random index: uint256(blockhash(block.number - 1)) % players.length
    // transfer whole balance to winner, emit WinnerSelected(winner, amount), reset the ro
und
```

In your report: explain honestly why blockhash-based randomness is **not safe** for real production lotteries (miner manipulation), and name one better alternative (e.g. Chainlink VRF).

Test: 5 accounts enter with exactly 0.01 ETH → wrong amount reverts → pickWinner → winner balance increases, others unchanged → a fresh round can start.

CONTRACT 10 — Time-Locked Vault (file: 10_TimeLockVault.sol)

LEVEL: Advanced | MARKS: 7 | EST. TIME: 1.5 hrs

Problem: Deposits are locked until a unlock time. A beneficiary can withdraw only after that time. Teaches timestamp logic and access control together.

```
address payable public beneficiary;
uint256 public unlockTime;
event Deposited(address indexed from, uint256 amount, uint256 newUnlockTime);
event Withdrawn(address indexed to, uint256 amount);

constructor(address payable _beneficiary, uint256 _lockSeconds) { ... }
    // unlockTime = block.timestamp + _lockSeconds
function deposit() external payable { ... }      // anyone, extends nothing
function extendLock(uint256 _extraSeconds) external onlyBeneficiary { ... }
function withdraw() external onlyBeneficiary { ... }
    // require block.timestamp >= unlockTime, 'Vault is still locked'
function timeRemaining() external view returns (uint256) { ... }
```

Test: deploy with 120 s lock → deposit 1 ETH → withdraw immediately → revert 'Vault is still locked' → wait 2 minutes (take a screenshot of the passing block timestamp!) → withdraw succeeds.

CONTRACT 11 — Escrow with Arbiter (file: 11_Escrow.sol)

LEVEL: Advanced | MARKS: 8 | EST. TIME: 2 hrs

Problem: A three-party escrow: buyer deposits, seller delivers, an arbiter resolves disputes. States must be enforced.

```
enum State { AwaitingPayment, AwaitingDelivery, Complete, Refunded, Disputed }
address payable public buyer;
address payable public seller;
address payable public arbiter;
State public state;

constructor(address payable _seller, address payable _arbiter) payable { ... }
    // buyer = msg.sender; ETH sent at deployment = the payment
function confirmDelivery() external onlyBuyer { ... }   // AwaitingDelivery -> Complete, p
ay seller
function refundBuyer() external onlyArbiterOrSeller { ... }
function raiseDispute() external onlyBuyerOrSeller { ... }  // -> Disputed
function resolveToSeller() external onlyArbiter { ... }     // only in Disputed
function resolveToBuyer() external onlyArbiter { ... }      // only in Disputed
// every function must revert if the current State is wrong -> guard with a stateCheck mod
ifier
```

Test (all 4 outcomes): (1) normal delivery → Complete; (2) seller refunds → Refunded; (3) dispute resolved to seller; (4) dispute resolved to buyer. Also prove that confirmDelivery in Disputed state reverts.

CONTRACT 12 — SimpleNFT (ERC-721 style) (file: 12_SimpleNFT.sol)

LEVEL: Advanced | MARKS: 8 | EST. TIME: 2.5 hrs

Problem: Your own NFT collection: mint unique token IDs, track owners, transfer between accounts. No imports — build it yourself.

```
string public name;
string public symbol;
uint256 public nextTokenId;
uint256 public constant MAX_SUPPLY = 100;
uint256 public mintPrice = 0.01 ether;
mapping(uint256 => address) public ownerOf;
mapping(address => uint256) public balances;
mapping(uint256 => address) public approvals;   // per-token approved spender
event Minted(address indexed to, uint256 indexed tokenId);
event TransferNFT(address indexed from, address indexed to, uint256 indexed tokenId);

function mint() external payable returns (uint256) { ... }
    // exact price, supply not exceeded, assign id = nextTokenId++
function transferNFT(address to, uint256 tokenId) external { ... }
    // only the owner can transfer
function approve(address spender, uint256 tokenId) external { ... }  // owner only
function transferFrom(address from, address to, uint256 tokenId) external { ... }
    // only the approved spender (mirrors ERC-721 flow)
function withdrawPayments() external onlyOwner { ... }  // owner withdraws mint revenue
```

Test: mint from Accounts 1–2 (pay exact price) → wrong payment reverts → Account 1 transfers token 0 to Account 3 → ownerOf(0) updated → Account 1 approves Account 2 for token 1 → Account 2 transferFrom works → owner withdraws 0.02 ETH of revenue.

CONTRACT 13 — English Auction (file: 13_EnglishAuction.sol)

LEVEL: Expert | MARKS: 9 | EST. TIME: 3 hrs

Problem: A timed auction: bidders outbid each other; losing bidders can withdraw their bids at any time; the seller ends the auction and receives the highest bid.

```
address payable public seller;
uint256 public auctionEnd;
address public highestBidder;
uint256 public highestBid;
uint256 public constant MIN_INCREMENT = 5;   // percent
mapping(address => uint256) public pendingReturns;  // refundable balances
bool public ended;
event NewBid(address indexed bidder, uint256 amount);
event AuctionEnded(address winner, uint256 amount);

constructor(uint256 _biddingSeconds, uint256 _startPrice) { ... }
function bid() external payable { ... }
    // auction not ended, not past auctionEnd,
    // msg.value >= highestBid * (100 + MIN_INCREMENT) / 100
    // previous highest bidder's money -> pendingReturns (do NOT send directly!)
function withdrawPending() external returns (bool) { ... }
function endAuction() external { ... }   // anyone may trigger after auctionEnd
    // pay seller, set ended, emit AuctionEnded. If nobody bid, mark ended only.
```

Key concept you must implement & explain: the **withdrawal (pull-payment) pattern** — losing bidders pull their own refunds, which removes the re-entrancy risk of pushing ETH inside bid(). Compare it with Contract 05 in your report.

Test: start price 0.1 ETH → A bids 0.11, B bids 0.12 (A gets pendingReturn) ↓ too-low bid 0.121 reverts (<5% increment) → A withdraws 0.11 ETH → wait for expiry → endAuction pays seller 0.12 → bid after end reverts.

CONTRACT 14 — Multi-Signature Wallet (file: 14_MultisigWallet.sol)

LEVEL: Expert | MARKS: 10 | EST. TIME: 3.5 hrs

Problem: A 2-of-3 style wallet: N owners, M confirmations required to execute any ETH transfer. This is how DAO treasuries and exchange cold wallets really work.

```
address[] public owners;
uint256 public required;                // e.g. 2 of 3
struct Transaction { address to; uint256 value; bool executed; }
Transaction[] public transactions;
mapping(uint256 => mapping(address => bool)) public confirmed;
event Submission(uint256 indexed txId);
event Confirmation(address indexed owner, uint256 indexed txId);
event Execution(uint256 indexed txId);
event Deposit(address indexed sender, uint256 amount);

constructor(address[] memory _owners, uint256 _required) { ... }
    // validate: _owners.length >= _required, no duplicates, no address(0)
receive() external payable { emit Deposit(msg.sender, msg.value); }
function submitTransaction(address to, uint256 value) external onlyOwner { ... }
function confirmTransaction(uint256 txId) external onlyOwner { ... }
    // one confirmation per owner; auto-execute when confirmations == required
function executeTransaction(uint256 txId) external onlyOwner { ... }
    // enough confirmations, not executed, call succeeds
function getConfirmations(uint256 txId) external view returns (uint256) { ... }
```

- Only owners may submit/confirm/execute (use a modifier with a loop or a mapping of valid owners).
- A transaction executes exactly once — guard with the executed flag.

- Track and expose the confirmation count per transaction.

  Test: deploy with owners [A, B, C], required = 2 → A submits a 1 ETH transfer to D → D receives nothing yet → B confirms → the transfer auto-executes → try confirming again from A → revert → non-owner E submits → revert. Verify D's balance +1 ETH in Ganache.

CONTRACT 15 — Staking with Time-Based Rewards (file: 15_StakingRewards.sol)

LEVEL: Expert | MARKS: 10 | EST. TIME: 4 hrs

Problem: The capstone. Users stake ETH and earn rewards continuously per second, like real staking platforms (Lido/Curve style). Combine mappings, math, timestamps, access control and safe transfers.

```
address payable public owner;         // funds the reward pool
uint256 public rewardRatePerSecond;   // wei of reward per staked wei per second
uint256 public totalStaked;
uint256 public rewardPool;
struct Stake { uint256 amount; uint256 startTime; uint256 rewardDebt; }
mapping(address => Stake) public stakes;

constructor(uint256 _rate) payable { ... }          // seed reward pool via msg.value
function stake() external payable { ... }
    // amount > 0; snapshot pending reward into rewardDebt BEFORE adding amount
function pendingReward(address user) public view returns (uint256) { ... }
    // rewardDebt + amount * rewardRatePerSecond * (block.timestamp - startTime)
function unstake() external { ... }
    // returns principal + full reward; resets the stake; one-shot safe
function claimReward() external { ... }              // reward only, stake keeps growing
function fundRewards() external payable onlyOwner { ... }   // top up pool
function setRewardRate(uint256 _newRate) external onlyOwner { ... }
```

- Reward math must use the **user's own stake start time** and handle multiple stakes/claims correctly.
- Guard against paying more than **rewardPool** — revert if the pool is empty.
- Apply Checks-Effects-Interactions everywhere.
- Optional (bonus): add a minimum staking duration of 1 day.

  Test: owner deploys with rate = 1000 wei/sec and 5 ETH pool → A stakes 1 ETH, B stakes 2 ETH → after ~60 s pendingReward(A) ≈ 60,000 wei (take an exact screenshot) → A claimReward → pending resets, stake intact → B unstakes → receives 2 ETH + rewards → B tries to unstake again → revert. Verify every wei in Ganache.

6. Testing & Documentation Requirements

For EACH of the 15 contracts you must capture minimum 2 screenshots (30 total):

- **Screenshot 1 — compile + deploy:** Remix showing the compiled contract AND the deployed instance with the Ganache-provided account address visible.
- **Screenshot 2 — happy path test:** the main function called successfully with its result visible in Remix, and/or the transaction visible in the Ganache TRANSACTIONS tab.
- **Extra screenshots (recommended):** revert/error cases — these show you actually tested edge cases and earn full marks.

REPORT.md — for every contract write 3–4 lines:

```
## 05 SimpleBank
- What it does : users deposit/withdraw ETH with per-user balances
- Key concept  : Checks-Effects-Interactions to block re-entrancy
- Tested       : deposit(2 ETH), withdraw(1 ETH), withdraw(10 ETH) -> reverted
- Ganache proof: screenshots/05_test1.png, 05_test2.png
```

Minimum testing checklist per contract: every public function called at least once ✓, at least one require-revert case triggered ✓, events verified in Ganache logs ✓, ETH balances verified in the Ganache ACCOUNTS tab ✓.

7. Final Submission (what you hand in)

You submit exactly **ONE thing**: the URL of your public GitHub repository, e.g.

```
https://github.com/your-username/solidity-smart-contracts-assignment
```

Pre-submission checklist — verify every box:

- ■ All 15 .sol files present in **contracts/**, correctly named, pragma ^0.8.x, all compile cleanly.
- ■ Every contract was deployed via **Custom External Http Provider** to **MY Ganache** (Remix VM testing alone = marks cut).
- ■ At least 2 screenshots per contract in **screenshots/** (30 minimum).
- ■ **README.md** filled with my name, roll no, and the status table.
- ■ **report/REPORT.md** completed for all 15 contracts.
- ■ Repository is **Public** and the last commit includes everything (check the file list on github.com).
- ■ I can explain any line of my code if asked.

Where to submit: paste your GitHub repo link in the assignment submission form / email / group as instructed by your instructor, with subject: **"Solidity Assignment — — "**. The instructor will clone/visit your repo and may re-run your contracts in Remix.

Late submissions: 10% marks deducted per day. Repositories made empty or private after the deadline are treated as not submitted.

8. Grading Rubric (100 Marks)

Component	Marks	What is checked
Contracts 01–05 (Basic/Medium)	20	Correct logic, compiles, requirements met
Contracts 06–10 (Medium/Advanced)	20	Structs, mappings, ETH handling, state logic
Contracts 11–15 (Advanced/Expert)	25	Enums, pull-payments, multisig, reward math

Ganache testing evidence	15	Transactions visible in Ganache, edge cases tested
Screenshots (min 30)	5	Clear, readable, showing addresses & results
README + REPORT.md	5	Complete, honest, matches the actual repo
GitHub usage	5	Proper commits (not 1 giant commit), correct structure
Code quality + viva	5	Comments, clean naming, can explain the code

Grade bands: 90+ = A | 80–89 = B | 70–79 = C | 60–69 = D | <60 = F (resubmit once)

9. Common Errors & Quick Fixes

Error in Remix / Ganache	Likely fix
Custom provider connection refused	Ganache not running, or wrong port — use http://127.0.0.1:7545
'sendTransaction' has reverted	A require() failed — read the error message in the Remix console
out of gas / stack too deep	Simplify function parameters; use structs or fewer local variables
Contract does not receive ETH	Function/contract missing the payable keyword
git push rejected	You forgot 'git pull' or the remote URL is wrong — check 'git remote -v'
GitHub asks for a password	Use a Personal Access Token (Section 3, Step 5) — not your password

Good luck — build every contract yourself, test it honestly on Ganache, and your GitHub repo will become the first real project on your blockchain portfolio.